

Prelab 5.1 MTConnect Adapter

Learning Goals

Students will be able to:

1. Start the MTConnect Adapter to examine contents via localhost.
2. Modify code to generate random values for multiple sensors in MTConnect
3. Demonstrate knowledge of the MTConnect and XML standards

1.1 Introduction

In the previous lab (Lab 4), we learned the concept of MTConnect and performed hands-on activities to run MTConnect agents in various MTConnect configurations. By MTConnect adapter simulation, we got to know the data stream from MTConnect adapter to MTConnect agent. In addition, we practiced the client request to MTConnect agent via HTTP request.

The purpose of Lab 5 is 1) to program MTConnect adapters in various configurations and 2) to be familiar with the client application (data collection via HTTP request and XML parsing). In this prelab, we will practice Python programming for MTConnect adapter and XML parsing. However, in Prelab 5, we will mainly practice and understand MTConnect adapter Python programming by generating random data.

1.2 MTConnect Adapter

In this prelab, we will program a simple Python code to run MTConnect adapter by generating random values without sensors. The schematic of MTConnect configuration is illustrated in Figure 1. The data item configuration is shown in Figure 2. The laptop plays the role of both MTConnect agent and MTConnect adapter. Before we collect data directly from the sensors (ADXL345, DS18B20, and virtual sensor --- there are no physical sensors related to humidity measurement), let's try to make random values assumed by the sensors as the MTConnect data transmission.

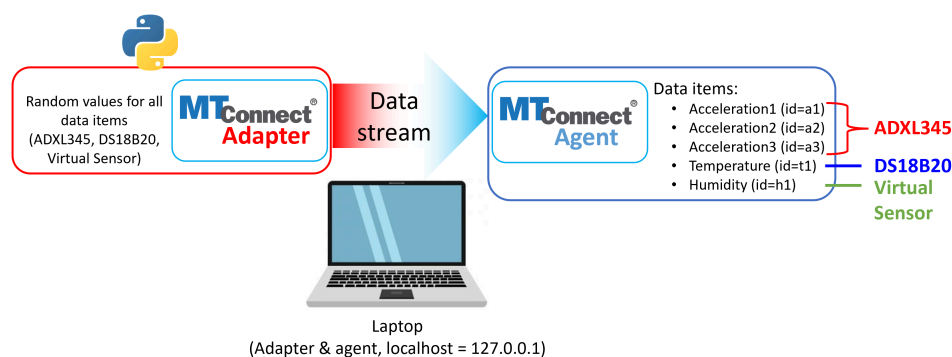


Figure 1 MTConnect Schematic

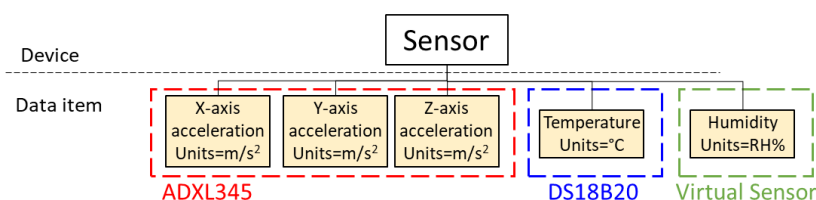


Figure 2 Data item configuration

For the MTConnect agent, you are able to reuse the complied execution file and other files from Lab 4. It would be better to make copy of the MTConnect agent directory every time rather than modify it every time in the original MTConnect agent. For the MTConnect agent set up, please follow procedure below.

1. Make the Host IP address and the Port of the 'Adapter1' as '127.0.0.1' and '7878', respectively
 - a. Again, 127.0.0.1 IP address means the machine host itself. That has the same meaning as localhost. Therefore, if you make 'Host' as 'localhost', it works as well.
 - b. Also, in the configuration file ('agent.cfg'), The '#' mark in front of the line means comment out the line as Python does. Therefore, MTconnect agent ignores the lines starting with '#'.
 - c. In this example, the Port number for the agent is 5000. But you can set a different port number.

```
10 Adapters {
11     # Log file has all machines with device name prefixed
12     Adapter1 {
13         Host = 127.0.0.1
14         Port = 7878
15     }
16 }
```

2. Make 'Device.xml' as the capture below (the same with Lab4 Task2.3, Task3.2, and Task3.3)

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <MTConnectDevices xmlns:m="urn:mtconnect.org:MTConnectDevices:1.1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns=
3 "urn:mtconnect.org:MTConnectDevices:1.1" xsi:schemaLocation="urn:mtconnect.org:MTConnectDevices:1.1
4 http://www.mtconnect.org/schemas/MTConnectDevices_1.1.xsd">
5   <Header creationTime="2010-03-04T18:44:40+00:00" sender="localhost" instanceId="1267728234" bufferSize="131072" version="1.1"
6   />
7   <Devices>
8     <Device id="s1" name="sensor" uuid="000">
9       <DataItems>
10         <DataItem category="EVENT" id="avail" type="AVAILABILITY"/>
11         <DataItem category="SAMPLE" id="a1" name="Xacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
12         <DataItem category="SAMPLE" id="a2" name="Yacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
13         <DataItem category="SAMPLE" id="a3" name="Zacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
14         <DataItem category="SAMPLE" id="h1" name="humd" type="HUMIDITY_RELATIVE" subType="ACTUAL" units="PERCENT"/>
15         <DataItem category="SAMPLE" id="t1" name="temp" type="TEMPERATURE" subType="ACTUAL" units="CELSIUS"/>
16       </DataItems>
17     </Device>
18   </Devices>
19 </MTConnectDevices>
```

3. Save 'agent.cfg' and 'Device.xml'
4. Run 'agent.exe' on 'Command Prompt'
5. Check that agent is working using 'http://127.0.0.1:5000/current' or 'http://localhost:5000/current' on a web browser.

If nothing is shown in your browser, please refer to 'Blank Screen Solution' in Task 2.1 of Lab 4: L4 Colab 2

- a. Because no adapters are working, the values of all data items are 'UNAVAILABLE'.

- creationTime: 2022-02-21T15:47:56Z - sender: DESKTOP-A8SR000 - instanceId: 1645458318 - version: 1.7.0.2 - bufferSize: 131072 - nextSequence: 9 - firstSequence: 1 - lastSequence: 8						
Device: sensor; UUID: 000						
Device : sensor						
Samples						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-02-21T15:45:18.103324Z	Acceleration		Xacc	a1	2	UNAVAILABLE
2022-02-21T15:45:18.103339Z	Acceleration		Yacc	a2	4	UNAVAILABLE
2022-02-21T15:45:18.103354Z	Acceleration		Zacc	a3	7	UNAVAILABLE
2022-02-21T15:45:18.103332Z	HumidityRelative	ACTUAL	humd	h1	3	UNAVAILABLE
2022-02-21T15:45:18.103357Z	Temperature	ACTUAL	temp	t1	8	UNAVAILABLE
Events						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-02-21T15:45:18.103206Z	Availability			avail	1	UNAVAILABLE
2022-02-21T15:45:18.103351Z	AssetChanged			s1_asset_chg	6	UNAVAILABLE
2022-02-21T15:45:18.103345Z	AssetRemoved			s1_asset_rem	5	UNAVAILABLE

Now, you are all set for the MTConnect agent part. Keep the agent running. **For the MTConnect adapter¹, please download [adapter_sample.zip](#) to laptop. Extract the zip file. The list on the zip file is below.**

- [adapter_sample.py](#)
 - Sample adapter program for this lab
 - There are comments on the lines for the hint and usage
- [data_item.py](#)
 - Data item package for MTConnect (by System Insights)
 - You do not need to touch this file, but if you are interested in details of program of MTConnect, please take look at it.
- [mtconnect_adapter.py](#)
 - MTConnect adapter package (by System Insights)
 - You do not need to touch this file, but if you are interested in details of program of MTConnect, please take look at it.

First, run the sample program ('*adapter_sample.py*') and then check '*current*' request to the MTConnect agent. The response will look like Figure 3. Because only X-axis acceleration (id = '*a1*') and temperature (id = '*t1*') values are added as the data items of MTConnect adapter and having the random values, other data items are still showing 'UNAVAILABLE'. When the MTConnect adapter is made a connection to the agent, the EVENT item id 'avail' is changed to 'AVAILABLE'. In reverse, the MTConnect adapter isn't made a connection to the agent, it is changed to 'UNAVAILABLE'. Therefore, by checking the 'Availability' (id = 'avail') we can see whether the adapter is connected to the agent well or not. If the adapter is connected but the SAMPLE values (sensor data) are not changing, we may consider the issues on the sensor data access. Please note that to halt (forcedly stop) the MTConnect adapter, the shortcut on the terminal is Ctrl + c. And the printing out values is to see what values are transmitting to the agent.

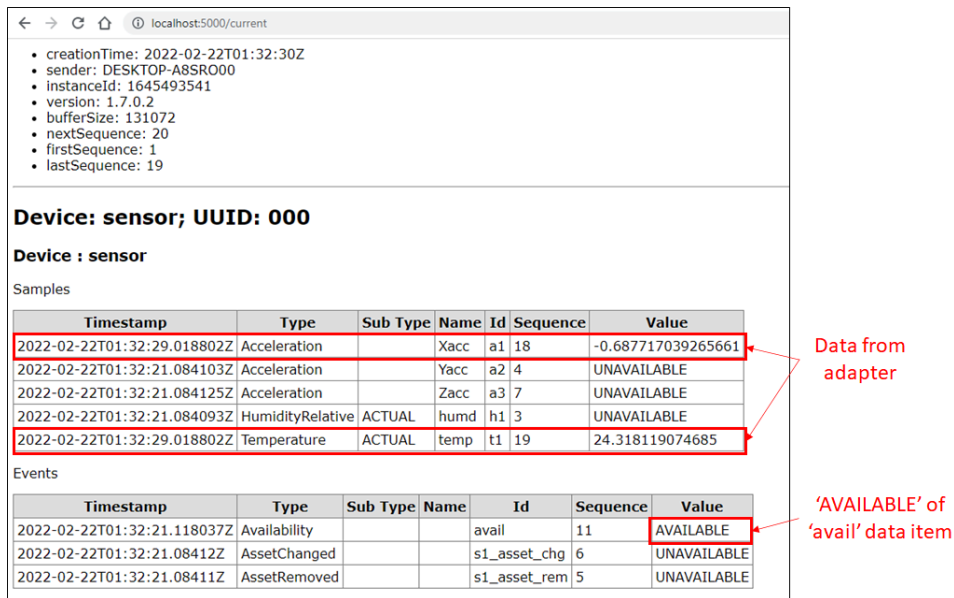


Figure 3 Capture of web browser of 'current' request to agent after running sample code

¹For this lab, MTConnect adapter based on Python was modified from https://github.com/mtconnect/PocketNC_adapter.

Task 1.1

By modifying the sample code ('adapter_sample.py'), generate random data streams for remaining data items (id = 'a2', 'a3', 'h1').

- Read the comments in the code.
- Make sure of the reasonable random value ranges.

Capture your web browser of 'current' request to agent after running the code as Figure 4 and attach it to the report below.

MTConnect Device Streams
127.0.0.1:5000/current

- creationTime: 2026-02-24T01:23:42Z
- sender: Daniel-PC
- instanceId: 1771895099
- version: 1.7.0.2
- bufferSize: 131072
- nextSequence: 16
- firstSequence: 1
- lastSequence: 15

Device: sensor; UUID: 000

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T01:23:42.440353Z	Acceleration		Xacc	a1	14	0.194407182030227
2026-02-24T01:04:59.107411Z	Acceleration		Yacc	a2	4	UNAVAILABLE
2026-02-24T01:04:59.107417Z	Acceleration		Zacc	a3	7	UNAVAILABLE
2026-02-24T01:04:59.107407Z	HumidityRelative	ACTUAL	humd	h1	3	UNAVAILABLE
2026-02-24T01:23:42.440353Z	Temperature	ACTUAL	temp	t1	15	24.0492152806341

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T01:23:39.027123Z	Availability		avail		11	AVAILABLE
2026-02-24T01:04:59.107416Z	AssetChanged		s1_asset_chg		6	UNAVAILABLE
2026-02-24T01:04:59.107413Z	AssetRemoved		s1_asset_rem		5	UNAVAILABLE

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <?xml-stylesheet type="text/xsl" href="/styles/Streams.xsl"?>
3 <MTConnectStreams xmlns:ms="urn:mtconnect.org:MTConnectStreams:1.5" xmlns="urn:mtconnect.org:MTConnectStreams:1.5" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   <Header creationTime="2026-01-07T22:10:45Z" sender="nabin-2" instanceId="1767823803" version="1.7.0.2" bufferSize="131072" nextSequence="120" firstSequence="1" lastSequence="15" />
5   <Streams>
6     <DeviceStream name="sensor" uuid="000">
7       <ComponentStream component="Device" name="sensor" componentId="s1">
8         <Samples>
9           <Acceleration dataItemId="a1" name="Xacc" sequence="115" timestamp="2026-01-07T22:10:43.90348Z">-0.164604491219</Acceleration>
10          <Acceleration dataItemId="a2" name="Yacc" sequence="116" timestamp="2026-01-07T22:10:43.90348Z">0.382444778817</Acceleration>
11          <Acceleration dataItemId="a3" name="Zacc" sequence="117" timestamp="2026-01-07T22:10:43.90348Z">-0.055562723153</Acceleration>
12          <HumidityRelative dataItemId="h1" name="humd" sequence="118" subtype="ACTUAL" timestamp="2026-01-07T22:10:43.90348Z">48.144459943754</HumidityRelative>
13          <Temperature dataItemId="t1" name="temp" sequence="119" subtype="ACTUAL" timestamp="2026-01-07T22:10:43.90348Z">22.359986367306</Temperature>
14        </Samples>
15        <Events>
16          <Availability dataItemId="avail" sequence="14" timestamp="2026-01-07T22:10:03.556585Z">AVAILABLE</Availability>
17          <AssetChanged dataItemId="s1_asset_chg" sequence="6" timestamp="2026-01-07T22:10:03.532721Z">UNAVAILABLE</AssetChanged>
18          <AssetRemoved dataItemId="s1_asset_rem" sequence="5" timestamp="2026-01-07T22:10:03.532715Z">UNAVAILABLE</AssetRemoved>
19        </Events>
20      </ComponentStream>
21    </DeviceStream>
22  </Streams>
23 </MTConnectStreams>
24

```

Figure 4 Capture of web browser of 'current' request to agent after running the modified sample code to have all random data

Task 1.2

It is possible to run two or more MTConnect adapters for an MTConnect agent. Try to split the MTConnect adapter into two adapters according to sensor types as Figure 5 then run both simultaneously. Follow the instructions and hints below.

- After performing this task, provide your codes in the code blocks below.
 - Provide your 'agent.cfg' in the text block below.
 - Confirm your adapters are working correctly from the MTConnect agent.
- Split the sample MTConnect Python code into two separate codes.
 - Filenames for the adapters are 'adapter_sample_Task1.2_1.py' and 'adapter_sample_Task1.2_2.py', respectively.
 - Make port numbers be 7878 and 7879, respectively.
 - Make each adapter have a different sampling period by changing time.sleep(t) method in the code. For example, sampling periods are 1 sec and 3 sec, respectively.
 - Hint: You don't need to change 'Device.xml'.

- Hint: You have to change 'agent.cfg'.
- Hint: You use the same IP address for two adapters.

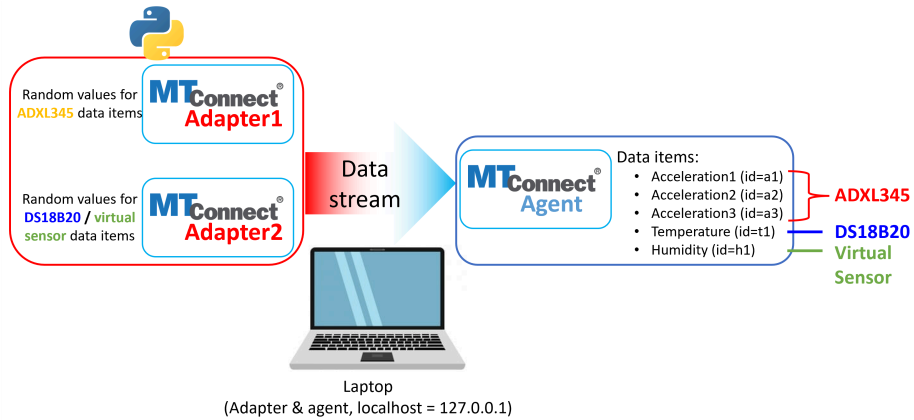


Figure 5 MTConnect schematic 2: two MTConnect adapters for an agent

```
In [ ]: ### Copy and paste 'adapter_sample_Task1.2_1.py' here

import random
import sys
import time
import datetime
from data_item import Event, Sample # Load data_item package
from mtconnect_adapter import Adapter # Load mtconnect_adapter package

class MTConnectAdapter(object):

    def __init__(self, host, port):
        # MTConnect adapter connection info
        self.host = host
        self.port = port
        self.adapter = Adapter((host, port))

        # For samples
        self.a1 = Sample("a1") # self.a1 takes 'a1' data item id.
        self.adapter.add_data_item(self.a1) # adding self.a1 as a data item
        self.a2 = Sample("a2") # self.a2 takes 'a2' data item id.
        self.adapter.add_data_item(self.a2) # adding self.a2 as a data item
        self.a3 = Sample("a3") # self.a3 takes 'a3' data item id.
        self.adapter.add_data_item(self.a3) # adding self.a3 as a data item
        ## Add more samples below

        # For events
        self.event = Event("event") # self.event takes 'event' data item name.
        self.adapter.add_data_item(
            self.event
        ) # adding self.event as a data item
        ## Add more events below

        # MTConnect adapter availability
        self.avail = Event("avail")
        self.adapter.add_data_item(self.avail)

        # Start MTConnect
        self.adapter.start()
        self.adapter.begin_gather()
        self.avail.set_value("AVAILABLE")
        self.adapter.complete_gather()
        self.adapter_stream()
```

```

def adapter_stream(self):
    while True:
        try:
            # Do something here.
            a1 = random.uniform(
                -1, 1
            ) # this example is to take a random float between -1 and 1.
            a2 = random.uniform(
                -1, 1
            ) # this example is to take a random float between -1 and 1.
            a3 = random.uniform(
                -1, 1
            ) # this example is to take a random float between -1 and 1.

            self.adapter.begin_gather()
            self.a1.set_value(
                str(a1)
            ) # set value of a1 data item, format: str(float)
            self.a2.set_value(
                str(a2)
            ) # set value of a2 data item, format: str(float)
            self.a3.set_value(
                str(a3)
            ) # set value of a3 data item, format: str(float)
            self.adapter.complete_gather()

            print(
                "{} RANDOM VALUE a1={} mm/s^2".format(
                    datetime.datetime.now(), a1
                )
            ) # printing out datetime now and a1
            print(
                "{} RANDOM VALUE a2={} mm/s^2".format(
                    datetime.datetime.now(), a2
                )
            ) # printing out datetime now and a2
            print(
                "{} RANDOM VALUE a3={} mm/s^2".format(
                    datetime.datetime.now(), a3
                )
            ) # printing out datetime now and a3
            print(
                datetime.datetime.now(),
                "MTConnect data items gathering completed...\n",
            ) # printing out MTConnect data collection is done.

            time.sleep(1) # wait for 2 seconds

        except KeyboardInterrupt:
            print("Stopping MTConnect...")
            self.adapter.stop() # Stop adapter thread
            sys.exit() # Terminate Python

## ===== MAIN =====
if __name__ == "__main__":
    print("Starting up!")
    MTConnectAdapter("127.0.0.1", 7878) # Args: host ip, port number

```

In []: `### Copy and paste 'adapter_sample_Task1.2_2.py' here`

```

import random
import sys
import time
import datetime
from data_item import Event, Sample # Load data_item package
from mtconnect_adapter import Adapter # Load mtconnect_adapter package

class MTConnectAdapter(object):

```

```

def __init__(self, host, port):
    # MTConnect adapter connection info
    self.host = host
    self.port = port
    self.adapter = Adapter((host, port))

    # For samples
    self.t1 = Sample("t1") # self.t1 takes 't1' data item id.
    self.adapter.add_data_item(self.t1) # adding self.t1 as a data item
    self.h1 = Sample("h1") # self.h1 takes 'h1' data item id.
    self.adapter.add_data_item(self.h1) # adding self.h1 as a data item
    ## Add more samples below

    # For events
    self.event = Event("event") # self.event takes 'event' data item name.
    self.adapter.add_data_item(
        self.event
    ) # adding self.event as a data item
    ## Add more events below

    # MTConnect adapter availability
    self.avail = Event("avail")
    self.adapter.add_data_item(self.avail)

    # Start MTConnect
    self.adapter.start()
    self.adapter.begin_gather()
    self.avail.set_value("AVAILABLE")
    self.adapter.complete_gather()
    self.adapter_stream()

def adapter_stream(self):
    while True:
        try:
            # Do something here.
            t1 = random.uniform(
                20, 25
            ) # this example is to take a random float between 15 and 25.
            h1 = random.uniform(
                40, 60
            ) # this example is to take a random float between 40 and 60.

            self.adapter.begin_gather()
            self.t1.set_value(
                str(t1)
            ) # set value of t1 data item, format: str(float)
            self.h1.set_value(
                str(h1)
            ) # set value of h1 data item, format: str(float)
            self.adapter.complete_gather()

            print(
                "{} RANDOM VALUE t1={} °C".format(
                    datetime.datetime.now(), t1
                )
            ) # printing out datetime now and t1
            print(
                "{} RANDOM VALUE h1={} %".format(
                    datetime.datetime.now(), h1
                )
            ) # printing out datetime now and h1
            print(
                datetime.datetime.now(),
                "MTConnect data items gathering completed...\n",
            ) # printing out MTConnect data collection is done.

            time.sleep(3) # wait for 2 seconds

        except KeyboardInterrupt:

```

```

        print("Stopping MTConnect...")
        self.adapter.stop() # Stop adapter thread
        sys.exit() # Terminate Python

## ===== MAIN =====
if __name__ == "__main__":
    print("Starting up!")
    MTConnectAdapter("127.0.0.1", 7879) # Args: host ip, port number

```

```

Devices = ./Device.xml
Port = 5000
AllowPut = true
ReconnectInterval = 1000
BufferSize = 17
MonitorConfigFiles = true
Pretty = true
SchemaVersion = 1.5

```

```

Adapters {
    # Log file has all machines with device name prefixed
    Adapter1 {
        Host = 127.0.0.1
        Port = 7878
    }
    Adapter2 {
        Host = 127.0.0.1
        Port = 7879
    }
}

```

```

Files {
    schemas {
        Path = ./schemas
        Location = /schemas/
    }
    styles {
        Path = ./styles
        Location = /styles/
    }
    Favicon {
        Path = ./styles/favicon.ico
        Location = /favicon.ico
    }
}

```

```

StreamsStyle {
    Location = /styles/Streams.xml
}

```

```

# Logger Configuration
logger_config
{
    logging_level = debug
    output = cout
}

```

• creationTime: 2026-02-24T01:48:12Z
 • sender: Daniel-PC
 • instanceId: 1771897195
 • version: 1.7.0.2
 • bufferSize: 131072
 • nextSequence: 1050
 • firstSequence: 1
 • lastSequence: 1049

Device: sensor; UUID: 000

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T01:48:12.419636Z	Acceleration		Xacc	a1	1047	-0.0871282686894628
2026-02-24T01:48:12.419636Z	Acceleration		Yacc	a2	1048	-0.705393246691184
2026-02-24T01:48:12.419636Z	Acceleration		Zacc	a3	1049	-0.753472787547387
2026-02-24T01:48:10.44388Z	HumidityRelative	ACTUAL	humd	h1	1043	58.1833088782164
2026-02-24T01:48:10.44388Z	Temperature	ACTUAL	temp	t1	1042	20.6028825590258

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T01:46:50.576085Z	Availability			avail	814	AVAILABLE
2026-02-24T01:39:55.345378Z	AssetChanged			s1_asset_chg	6	UNAVAILABLE
2026-02-24T01:39:55.345375Z	AssetRemoved			s1_asset_rem	5	UNAVAILABLE

Additional discussion

In this prelab, we only used a laptop for MTConnect agent as well as adapter. By using the sample adapter code, can you create different configurations for MTConnect? For example, Raspberry Pi is an MTConnect adapter and laptop is an MTConnect agent. Justify and describe how you make the MTConnect architecture below. You can also use figures, pictures, codes, and so on.

Yes, you can create different configurations for MTConnect. For example, since you can specify a configuration file at runtime, you could have a library full of different agent configurations. When using windows for example:

```
agent.exe run agent1.cfg
```

This will specify to use a configuration file named `agent1.cfg`.

You can do the same thing with different adapters by writing separate python files for each adapter and running them in separate command windows. You could also use subprocesses to run different adapters simultaneously or design an adapter two run asynchronously and then run two different instances.

The overall architecture for creating different configurations would be:

- Raspberry pi: Run multiple adapters either through subprocesses, separate windows or async programming
- Windows: Run multiple agents using different configuration files.

MTConnect Standard

In Lab 5, we will run the MTConnect adapter for each sensor on Raspberry Pi. Please take a look at the PDF document linked below and read the following chapters:

- (Reading) MTConnect® Standard, Part 1.0 – Overview and Fundamentals, Version 1.8.0:
https://docs.mtconnect.org/MTConnect_Part_1-0-Overview_and_Fundamentals_1-8-0.pdf
 - Chapter 6 XML representation of Response Documents
 - Chapter 7 Protocol and Messaging

- Chapter 8 HTTP Messaging Supported by an Agent

Task 1.3

Answer the questions below in response to the MTConnect Standard document above.

1. Find and provide the time-zone that MTConnect timestamp uses and explain why it is that time-zone.

The time-zone that MTConnect uses is the UTC/GMT or Coordinated Universal Time/Greenwich Mean Time. MTConnect uses this timestamp because it is the primary time standard by which the world regulates clocks and time.

2. List the primary components necessary between an MTConnect agent and client application.

The primary components necessary between an MTConnect Agent and Client application are requests and responses. Client software sends a request to an agent, typically over HTTP. In return, the agent will publish a response document. This response document will include a header and devices element, where the header will define version information, storage capacity and other parameters associated with the agent. The Devices element contains the equipment metadata for example sensor readings.

3. What is the interface that an MTConnect agent must use, that is supported by HTTP?

The MTConnect agent uses the REST interface to communicate with client applications. The REST interface supports an Application Programming Interface with which to retrieve data from devices.

XML Parsing and XPath

In Lab 5, we will also explore MTConnect data deeper. For that, we need to know and be familiar with XML parsing and XPath. By using XPath and XML parsing, you can fluently handle the MTConnect request and responded XML documents.

- (Tutorial) XML parsing in Python
 - GeeksforGeeks: <https://www.geeksforgeeks.org/xml-parsing-python/>
 - Python Standard Library: <https://docs.python.org/3/library/xml.etree.elementtree.html>
- (Tutorial) XPath
 - W3Schools: https://www.w3schools.com/xml/xpath_intro.asp

Task 1.4

Answer the questions below in response to the XML content above.

1. What is XML parsing? Define XML parsing and explain how it works.

XML parsing is the process of breaking up an XML file into usable objects that another software can use. For example in python, using the python standard library you can extract all of the data contained within an xml file. It works by using parent-child relationships which can be iterated over to extract data. All of the child nodes within a parent tag can be extracted, and the data itself is contained within its attributes.

2. What is XPath?

XPath is a way to convert an XML document into a more familiar format of the pathing. It is also a major element in the eXtensible Stylesheet Language Transformation (XSLT) standard. This allows data any XML document to be accessed through simple string expressions. This allows parsers to be built very easily in any language.

Get back to [Lab Index Page](#)